# Transfer List (move items between 2 lists)

**Problem:** Build a transfer list: two lists side by side, items can be checked and moved between them with "move selected" and "move all" buttons.

**Often asked at:** Meta, Atlassian · **Core skill tested:** set operations on state

**Tech stack:** React 19 (Vite, JavaScript, plain CSS) · **Time limit:** 60 minutes (defaults)

---

## PART A: Thought process

### The general steps for any machine coding task

1. **Understand the requirements**: what it must do, what "good" means, what is out of scope. Ask before you build.
2. **List the interactions**: every action a user can take and what should happen. This is your checklist.
3. **Design the architecture**: components and their single responsibility.
4. **Design the state**: the minimum state, who owns it, what is derived.
5. **Design data structures and contracts**: shapes of data, props, actions.
6. **Trace one data flow end to end**: proves the design before code.
7. **Find the hard parts**: solve them on paper first.
8. **Build in layers**: simplest working version first, then one working layer at a time.
9. **Cover states and edge cases**: empty, error, disabled, boundaries.
10. **Wrap up**: explain trade-offs and what you would add with more time.

### Time budget for 60 minutes

| Phase | Time |
|---|---|
| Requirements + clarifying questions | 5 min |
| Interactions, architecture, state design | 8 min |
| Core build: render lists, toggle, move selected | 20 min |
| Move all, disabled buttons, select-all + indeterminate | 12 min |
| Search, empty states, a11y polish | 10 min |
| Wrap-up, trade-offs, questions | 5 min |

### Tip: think out loud

Say every decision and its reason as you make it: *"I'll keep both lists and the checked ids in one reducer, because a move changes three things at once and they must never be out of sync."* The interviewer grades your reasoning as much as the result. If you get stuck, say what you are trying and what you would check next.

---

# PART B: Apply it to this problem

## 1. Understand the requirements

### Functional requirements

- Show two lists side by side: **left (Available)** and **right (Selected)**.
- Each item has a checkbox; clicking the item or the checkbox toggles it.
- Four buttons between the lists:
  - `≫` move **all** items left → right
  - `>` move **checked** items left → right
  - `<` move **checked** items right → left
  - `≪` move **all** items right → left
- A button is **disabled** when it has nothing to move.
- Moved items land at the end of the other list and arrive **unchecked**.
- Each list has a **select-all** checkbox (with an indeterminate state when partly selected) and a "x/y selected" count.
- Each list has a **search box** that filters only that list.

### Non-functional requirements

| Area | What it means here |
|---|---|
| Performance | Moves are O(n) with Set lookups, not O(n·m) with `includes`. Fine for hundreds of items; virtualize for 10k+. |
| Scalability | Item shape is `{ id, label }`; any data source can be mapped into it. |
| Error handling | No network, so "errors" are invalid actions: moving from an empty list or with nothing checked must be a safe no-op. |
| Accessibility | Real `<input type="checkbox">` inside `<label>`, `aria-label` on icon buttons (`>` means nothing to a screen reader), `aria-live` count, visible focus, keyboard works with Tab/Space/Enter. |
| Usability | Disabled buttons show what is possible; empty and "no matches" messages; checked rows highlighted. |
| Reusability | `TransferList` takes `initialLeft` / `initialRight` props; the panel component is used twice. |
| Extensibility | All logic is in a pure reducer, so new actions (reorder, add item, persist) are new cases, not rewrites. |

### Clarifying questions to ask the interviewer

1. Should moved items keep their **original order**, or be **appended** to the end of the target list?
2. After a move, should the moved items stay **checked** or become unchecked?
3. Do we need **search** inside each list? If so, does "select all" mean all items or only the visible (filtered) ones?
4. Is the data **static** or loaded from an **API**? Do moves need to be saved to a server?
5. Do we need **drag and drop** or **keyboard shortcuts** (Shift-click range select)?
6. Should the component report changes to a **parent** (controlled `value` / `onChange`)?

### Assumptions we will build with

- Data is static, passed in as props; no API calls.
- Moved items are **appended** to the end of the target list and arrive **unchecked**.
- **Select-all acts on visible items only** (what the user can see is what they select).
- **Move selected moves every checked item in that list**, including ones hidden by the search. The header count "3/8 selected" makes hidden selections visible.
- Items have unique, stable `id`s. Labels may repeat.
- No drag and drop and no persistence in the 60-minute version.

---

## 2. Identify interactions / use cases

| Action / input | Expected behavior / output |
|---|---|
| Page loads | Left shows 6 items, right shows 2, nothing checked. `>` and `<` disabled, `≫` and `≪` enabled. Counts show `0/6` and `0/2`. |
| Click an item (label or checkbox) | Item toggles checked, row highlights, count updates, matching `>` or `<` becomes enabled. |
| Click the same item again | Item unchecks; button disables again if it was the only checked item. |
| Check some (not all) items in a list | Select-all checkbox shows **indeterminate** (–). |
| Click select-all when none / some checked | All **visible** items in that list become checked. |
| Click select-all when all visible checked | All visible items in that list uncheck. |
| Click `>` | Checked left items are removed from left, appended to right in their left order, and unchecked. Right's checked items are untouched. |
| Click `≫` | Every left item moves to the right (checked or not); left shows "No items"; `≫` and `>` disable. |
| Click `<` / `≪` | Mirror of `>` / `≫` from right to left. |
| Type in a list's search | Only that list filters (case-insensitive, by label). Checked state of hidden items is kept. |
| Search has no matches | Shows `No matches for "xyz"`; select-all disabled. |
| Select-all while filtered | Only the visible matches are checked. |
| Click `>` while filtered with hidden checked items | All checked items in left move (count told the user). |
| List becomes empty | Shows "No items", select-all disabled and unchecked, "move all" from it disabled. |
| Keyboard: Tab / Space / Enter | Tab reaches every checkbox, search and button; Space toggles a checkbox; Enter/Space presses a button. Disabled buttons are skipped. |

This table is the implementation checklist and the test list.

---

## 3. Design the architecture

| Component / module | Responsibility |
|---|---|
| `App` | Page shell; passes initial data. |
| `TransferList` | **Owns the state** ( `useReducer` ): both lists + checked ids. Derives counts. Wires panels and controls. |
| `TransferPanel` | Renders one list: header (select-all, title, count), search, items, empty state. Owns only its own search text. Used twice. |
| `TransferControls` | The four buttons, their disabled rules, and calling `onMove` . |
| `transferReducer.js` | Pure functions: `TOGGLE` , `SET_MANY` , `MOVE` . No React, easy to test. |
| `data/items.js` | Example data with stable ids. |

```
App
└── TransferList            (state: left, right, checked)
    ├── TransferPanel  "Available"   (local: query)
    ├── TransferControls             (≫ > < ≪)
    └── TransferPanel  "Selected"    (local: query)

transferReducer.js  ← pure logic used by TransferList
```

**Start simple:** first version is one `TransferList` file with two hard-coded `<ul>` s and four buttons. Split out `TransferPanel` as soon as you notice you wrote the same list markup twice, and `TransferControls` when the button rules grow.

---

## 4. Design the state / data model

| State or field | Purpose | Owner |
|---|---|---|
| `left: Item[]` | Items in the left list, in display order | `TransferList` (reducer) |
| `right: Item[]` | Items in the right list, in display order | `TransferList` (reducer) |
| `checked: Set<id>` | Ids of checked items, across **both** lists | `TransferList` (reducer) |
| `query: string` | Search text for one panel | `TransferPanel` ( `useState` ) |

```
// One reducer state: a move changes left, right and checked together.
const [state, dispatch] = useReducer(
  transferReducer,
```

```
  { left: initialLeft, right: initialRight },
  createInitialState, // → { left, right, checked: new Set() }
);
```

### Senior note: derived vs stored, and ownership

- **Derived, not stored:** `leftChecked`, `rightChecked` (counts), button `disabled` flags, the filtered `visible` list, `allVisibleChecked` and `someVisibleChecked` (select-all state). Storing any of these creates a second source of truth that can drift after a move.
- **One `checked` Set, not two:** ids are unique across both lists, so "is it checked in the left list" is `left ∩ checked`. One set means a move only has to *remove* ids; it never has to copy them between sets.
- **Why a `Set`:** `has` is O(1). With an array, `left.filter(i => checked.includes(i.id))` is O(n·m).
- **Search is local to each panel:** no other component needs it, so lifting it up would only cause extra re-renders and props.
- **Why `useReducer` over three `useState`s:** a move is one event that updates three values. One reducer makes it atomic, pure and testable without rendering.

---

## 5. Design the data structures and contracts

### Example data

```
// Stable ids: labels can repeat or change, ids never do.
export const INITIAL_LEFT = [
  { id: 'js', label: 'JavaScript' },
  { id: 'ts', label: 'TypeScript' },
  { id: 'py', label: 'Python' },
  { id: 'go', label: 'Go' },
  { id: 'rs', label: 'Rust' },
  { id: 'java', label: 'Java' },
];
export const INITIAL_RIGHT = [
  { id: 'kt', label: 'Kotlin' },
  { id: 'swift', label: 'Swift' },
];
```

**Why ids:** they are the React `key` (rows keep their DOM and focus when lists change), the key in the `checked` Set, and they let two items share a label ("Java" from two teams) without breaking selection.

### Component contracts

| Component | Props in | Out (callbacks) |
|---|---|---|
| `TransferList` | `initialLeft: Item[]`, `initialRight: Item[]` | – (future: `onChange({ left, right })`) |

| Component | Props in | Out (callbacks) |
|---|---|---|
| `TransferPanel` | `title`, `items: Item[]`, `checked: Set`, `dispatch` | dispatches `TOGGLE`, `SET_MANY` |
| `TransferControls` | `leftCount`, `rightCount`, `leftChecked`, `rightChecked` | `onMove(from, to, all)` |

### Reducer action contracts

```
{ type: 'TOGGLE',   id: 'py' }                                  // flip one item
{ type: 'SET_MANY', ids: ['js', 'ts'], value: true }            // select-all / clear visible
{ type: 'MOVE',     from: 'left', to: 'right', all: false }      // move checked
{ type: 'MOVE',     from: 'right', to: 'left', all: true }       // move all
```

### If there were an API (mock contract)

```
GET  /api/transfer        → 200 { "left": [{ "id": "js", "label": "JavaScript" }], "right": [] }
PUT  /api/transfer        body { "rightIds": ["js", "py"] }  → 200 { "ok": true }
```

Send only ids to the server; it already knows the labels.

---

## 6. Trace the data flow

**Interaction: user checks "Python" and "Rust" in the left list, then clicks `>`.**

```
Click "Python" label
   │
   ▼
TransferPanel  dispatch({ type: 'TOGGLE', id: 'py' })
   │
   ▼
transferReducer  checked = new Set(old) + 'py'        (new Set → new reference)
   │   (same for "Rust" → checked = { py, rs })
   ▼
TransferList re-renders → derives leftChecked = 2
   │
   ├─► TransferPanel(left): rows highlighted, count "2/6 selected",
   │                        select-all becomes indeterminate (effect sets DOM property)
   └─► TransferControls: ">" enabled (leftChecked > 0)

Click ">"
   │
   ▼
TransferControls  onMove('left', 'right', false)
```

```
   │
   ▼
dispatch({ type: 'MOVE', from: 'left', to: 'right', all: false })
   │
   ▼
transferReducer
   moving    = left ∩ checked            → [Python, Rust]
   left      = left − moving             → [JS, TS, Go, Java]
   right     = right + moving            → [Kotlin, Swift, Python, Rust]
   checked   = checked − moving          → {}
   │
   ▼
TransferList re-renders → leftChecked = 0, rightChecked = 0
   │
   ├─► Left panel: 4 items, "0/4 selected", select-all unchecked
   ├─► Right panel: 4 items, Python and Rust at the end, unchecked
   └─► Controls: ">" and "<" disabled again
```

---

## 7. Important technical challenges

### Challenge 1: Moving items atomically (the set operations)

**Problem:** a move changes three things: the source list, the target list and the checked ids. With three `useState`s, a naive version looks like this:

```
setRight([...right, ...left.filter((i) => checked.includes(i.id))]);
setLeft(left.filter((i) => !checked.includes(i.id)));
// forgot to clear checked → moved items show up already checked on the right,
// and "<" is enabled right after ">" was clicked.
```

Scenario: the user double-clicks `>` fast. If the second handler reads a stale `left`, the same items get appended to `right` **twice** (duplicate keys, React warnings, a "ghost" item).

**Solution:** one pure reducer computes the whole next state from the current state, as set operations: `moving = A ∩ checked`, `A = A − moving`, `B = B + moving`, `checked = checked − moving`. React always passes the latest state to the reducer, so a second click finds nothing checked and returns the same state (no-op, no re-render).

```
case 'MOVE': {
  const { from, to, all } = action;
  const moving = all ? state[from] : state[from].filter(inSet(state.checked));
  if (moving.length === 0) return state; // no-op keeps the same reference

  const movingIds = new Set(moving.map((item) => item.id));
  const checked = new Set(state.checked);
```

```
  movingIds.forEach((id) => checked.delete(id));

  return {
    ...state,
    [from]: state[from].filter(notInSet(movingIds)), // A − moving
    [to]: [...state[to], ...moving],                 // B + moving
    checked,                                         // moved items arrive unchecked
  };
}
```

One `MOVE` action serves all four buttons, so there is one code path to test instead of four.

### Challenge 2: Immutable updates to a `Set`

**Problem:** `checked.add(id); setChecked(checked)` mutates the Set in place. The reference is the same, so React bails out and **nothing re-renders**. The checkbox looks dead.

**Solution:** copy, then change, then return the copy.

```
case 'TOGGLE': {
  const checked = new Set(state.checked);
  if (checked.has(action.id)) checked.delete(action.id);
  else checked.add(action.id);
  return { ...state, checked };
}
```

The same rule applies to the lists: `filter` and spread create new arrays; never `splice` or `push` on state.

### Challenge 3: Select-all with an indeterminate state, combined with search

**Problem:** the header checkbox has three looks: none, all, some. HTML has no `indeterminate` **attribute**, only a DOM **property**, so `<input indeterminate />` does nothing in React. Also, with a search active, what does "all" mean? Scenario: left has 6 items, search "a" shows 3 (JavaScript, TypeScript, Java). If select-all checks all 6, the user silently selects 3 items they cannot see.

**Solution:** derive the three states from **visible** items only, and set the DOM property with a ref.

```
const visibleChecked = visible.filter((item) => checked.has(item.id)).length;
const allVisibleChecked = visible.length > 0 && visibleChecked === visible.length;
const someVisibleChecked = visibleChecked > 0 && !allVisibleChecked;

useEffect(() => {
  selectAllRef.current.indeterminate = someVisibleChecked;
}, [someVisibleChecked]);

const toggleAllVisible = () =>
  dispatch({ type: 'SET_MANY', ids: visible.map((i) => i.id), value: !allVisibleChecked });
```

`SET_MANY` only adds or removes the given ids, so hidden checked items are never touched. The `visible.length > 0` guard stops an empty list from showing as "all checked" ( `0 === 0` ).

### Challenge 4: Correct disabled states and accessible icon buttons

**Problem:** `>` with nothing checked does nothing, which feels broken. And a screen reader reads `>` as "greater than".

**Solution:** derive each button's `disabled` from counts, keep the rules in one config array, and give every button an `aria-label` .

```
const buttons = [
  { text: '≫', label: 'Move all to right',      from: 'left',  to: 'right', all: true,  disabled:
leftCount === 0 },
  { text: '>', label: 'Move selected to right', from: 'left',  to: 'right', all: false, disabled:
leftChecked === 0 },
  { text: '<', label: 'Move selected to left',  from: 'right', to: 'left',  all: false, disabled:
rightChecked === 0 },
  { text: '≪', label: 'Move all to left',       from: 'right', to: 'left',  all: true,  disabled:
rightCount === 0 },
];
```

The reducer still guards `moving.length === 0` , so the logic is safe even if the UI rule is wrong (defense in depth).

---

## 8. Implement in the correct order

| # | Step | Working result after the step |
|---|---|---|
| 1 | Static layout: two `<ul>` s and four buttons, flex row, hard-coded items | Looks right, does nothing |
| 2 | Data file with ids + `useReducer` with `createInitialState` ; render lists from state | Lists come from state |
| 3 | `TOGGLE` action + checkbox per row inside `<label>` | Items can be checked |
| 4 | `MOVE` (selected) wired to `>` and `<` | Core feature works |
| 5 | `MOVE` with `all: true` for `≫` and `≪` | All four buttons work |
| 6 | Derived counts + `disabled` on buttons | No dead clicks |
| 7 | Extract `TransferPanel` and `TransferControls` (same markup was written twice) | Cleaner code, same behavior |
| 8 | Select-all checkbox: `SET_MANY` + indeterminate via ref | Bulk selection |
| 9 | Search per panel (local `query` , derived `visible` ); select-all acts on visible | Filtering |
| 10 | Empty and "no matches" states, "x/y selected" count | All states shown |

| # | Step | Working result after the step |
|---|---|---|
| 11 | A11y: `aria-label`s, `aria-live` count, focus styles, mobile stacking | Polished |
| 12 | Reducer self-check with node `assert` | Logic verified |

If time runs out after step 6, you still have a complete, correct transfer list.

---

## 9. Cover all states and edge cases

| State / edge case | What the UI does |
|---|---|
| Initial | Both lists rendered, nothing checked, only "move all" buttons enabled (if lists are non-empty). |
| Some items checked | Rows highlighted, select-all indeterminate, count "2/6 selected", matching "move selected" enabled. |
| All visible checked | Select-all checked (not indeterminate). |
| After a move (success) | Items appear at the end of the target list, unchecked; both counts and buttons update. |
| Source list empty | "No items", select-all disabled, both its buttons disabled. |
| Both lists empty | All four buttons disabled, both panels show "No items". |
| Search with no matches | `No matches for "xyz"`, select-all disabled. |
| Search active + hidden checked items | Count still includes hidden checked items; move moves them too (documented assumption). |
| Search cleared | Full list back, checked state preserved. |
| Click a disabled button | Nothing happens (browser ignores it; reducer no-op guard as backup). |
| Rapid double click on `>` | Second dispatch finds nothing checked → same state returned, no duplicates. |
| Duplicate labels | Fine: identity is `id`, not `label`. |
| Item moved back and forth | Same `id`, so React keeps keys stable; no duplicates. |
| Very long label | Text wraps inside the panel (`min-width: 0` on flex child prevents overflow). |
| Many items (1000+) | Panel scrolls (`overflow-y: auto`); Set lookups keep moves O(n). Virtualize beyond ~5k. |
| Narrow screen | Panels stack vertically, buttons go into a row. |
| Loading / error | Not applicable (static data). With an API: skeleton rows while loading, retry message on error, buttons disabled until loaded. |

---

## 10. Final structure

```
_68_transfer_list/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── data/
    │   └── items.js
    ├── utils/
    │   └── transferReducer.js
    └── components/
        ├── TransferList.jsx
        ├── TransferPanel.jsx
        └── TransferControls.jsx
```

## 11. Full working code

**`src/main.jsx`**

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

**`src/App.jsx`**

```
import TransferList from './components/TransferList';
import { INITIAL_LEFT, INITIAL_RIGHT } from './data/items';
import './index.css';

export default function App() {
  return (
    <main className="app">
      <h1>Transfer List</h1>
```

```
      <TransferList initialLeft={INITIAL_LEFT} initialRight={INITIAL_RIGHT} />
    </main>
  );
}
```

**src/data/items.js**

```
// Stable ids: labels can repeat or change, ids never do.
export const INITIAL_LEFT = [
  { id: 'js', label: 'JavaScript' },
  { id: 'ts', label: 'TypeScript' },
  { id: 'py', label: 'Python' },
  { id: 'go', label: 'Go' },
  { id: 'rs', label: 'Rust' },
  { id: 'java', label: 'Java' },
];

export const INITIAL_RIGHT = [
  { id: 'kt', label: 'Kotlin' },
  { id: 'swift', label: 'Swift' },
];
```

**src/utils/transferReducer.js**

```
// Pure state logic: no React, so it is easy to unit test.
export function createInitialState({ left = [], right = [] }) {
  return { left, right, checked: new Set() };
}

const inSet = (set) => (item) => set.has(item.id);
const notInSet = (set) => (item) => !set.has(item.id);

export function countChecked(list, checked) {
  return list.reduce((n, item) => (checked.has(item.id) ? n + 1 : n), 0);
}

export function transferReducer(state, action) {
  switch (action.type) {
    case 'TOGGLE': {
      const checked = new Set(state.checked);
      if (checked.has(action.id)) checked.delete(action.id);
      else checked.add(action.id);
      return { ...state, checked };
    }

    // Select all / clear all for the ids the panel currently shows.
```

```
    case 'SET_MANY': {
      const checked = new Set(state.checked);
      action.ids.forEach((id) => (action.value ? checked.add(id) : checked.delete(id)));
      return { ...state, checked };
    }

    // from/to: 'left' | 'right'; all: move every item, not only checked ones.
    case 'MOVE': {
      const { from, to, all } = action;
      const moving = all ? state[from] : state[from].filter(inSet(state.checked));
      if (moving.length === 0) return state;

      const movingIds = new Set(moving.map((item) => item.id));
      const checked = new Set(state.checked);
      movingIds.forEach((id) => checked.delete(id));

      return {
        ...state,
        [from]: state[from].filter(notInSet(movingIds)), // A - moving
        [to]: [...state[to], ...moving], //                B + moving
        checked, //                                        moved items arrive unchecked
      };
    }

    default:
      return state;
  }
}
```

**src/components/TransferList.jsx**

```
import { useReducer } from 'react';
import { createInitialState, transferReducer, countChecked } from '../utils/transferReducer';
import TransferPanel from './TransferPanel';
import TransferControls from './TransferControls';

export default function TransferList({ initialLeft, initialRight }) {
  const [state, dispatch] = useReducer(
    transferReducer,
    { left: initialLeft, right: initialRight },
    createInitialState,
  );
  const { left, right, checked } = state;

  // Derived, never stored: recomputed from left/right/checked on every render.
  const leftChecked = countChecked(left, checked);
  const rightChecked = countChecked(right, checked);
```

```
  return (
    <div className="transfer">
      <TransferPanel title="Available" items={left} checked={checked} dispatch={dispatch} />
      <TransferControls
        leftCount={left.length}
        rightCount={right.length}
        leftChecked={leftChecked}
        rightChecked={rightChecked}
        onMove={(from, to, all) => dispatch({ type: 'MOVE', from, to, all })}
      />
      <TransferPanel title="Selected" items={right} checked={checked} dispatch={dispatch} />
    </div>
  );
}
```

**src/components/TransferPanel.jsx**

```
import { useEffect, useId, useMemo, useRef, useState } from 'react';

export default function TransferPanel({ title, items, checked, dispatch }) {
  const [query, setQuery] = useState(''); // UI-only state: lives in the panel
  const titleId = useId();
  const selectAllRef = useRef(null);

  const visible = useMemo(() => {
    const q = query.trim().toLowerCase();
    return q ? items.filter((item) => item.label.toLowerCase().includes(q)) : items;
  }, [items, query]);

  const checkedCount = items.filter((item) => checked.has(item.id)).length;
  const visibleChecked = visible.filter((item) => checked.has(item.id)).length;
  const allVisibleChecked = visible.length > 0 && visibleChecked === visible.length;
  const someVisibleChecked = visibleChecked > 0 && !allVisibleChecked;

  // `indeterminate` is a DOM property only, there is no HTML attribute for it.
  useEffect(() => {
    selectAllRef.current.indeterminate = someVisibleChecked;
  }, [someVisibleChecked]);

  const toggleAllVisible = () =>
    dispatch({ type: 'SET_MANY', ids: visible.map((item) => item.id), value: !allVisibleChecked });

  return (
    <section className="panel" aria-labelledby={titleId}>
      <header className="panel__header">
        <label className="panel__select-all">
```

```
          <input
            ref={selectAllRef}
            type="checkbox"
            checked={allVisibleChecked}
            disabled={visible.length === 0}
            onChange={toggleAllVisible}
            aria-label={`Select all in ${title}`}
          />
          <span id={titleId} className="panel__title">{title}</span>
        </label>
        <span className="panel__count" aria-live="polite">
          {checkedCount}/{items.length} selected
        </span>
      </header>

      <input
        type="search"
        className="panel__search"
        placeholder="Search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        aria-label={`Search ${title}`}
      />

      {visible.length === 0 ? (
        <p className="panel__empty">{items.length === 0 ? 'No items' : `No matches for "${query}"`}</p>
      ) : (
        <ul className="panel__list">
          {visible.map((item) => (
            <li key={item.id}>
              <label className={`panel__item ${checked.has(item.id) ? 'is-checked' : ''}`}>
                <input
                  type="checkbox"
                  checked={checked.has(item.id)}
                  onChange={() => dispatch({ type: 'TOGGLE', id: item.id })}
                />
                {item.label}
              </label>
            </li>
          ))}
        </ul>
      )}
    </section>
  );
}
```

**src/components/TransferControls.jsx**

```
export default function TransferControls({ leftCount, rightCount, leftChecked, rightChecked, onMove })
{
  // Config array: one place to read every button's rule.
  const buttons = [
    { text: '≫', label: 'Move all to right', from: 'left', to: 'right', all: true, disabled: leftCount
=== 0 },
    { text: '>', label: 'Move selected to right', from: 'left', to: 'right', all: false, disabled:
leftChecked === 0 },
    { text: '<', label: 'Move selected to left', from: 'right', to: 'left', all: false, disabled:
rightChecked === 0 },
    { text: '≪', label: 'Move all to left', from: 'right', to: 'left', all: true, disabled: rightCount
=== 0 },
  ];

  return (
    <div className="controls" role="group" aria-label="Transfer actions">
      {buttons.map((b) => (
        <button
          key={b.label}
          type="button"
          className="controls__btn"
          disabled={b.disabled}
          aria-label={b.label}
          title={b.label}
          onClick={() => onMove(b.from, b.to, b.all)}
        >
          {b.text}
        </button>
      ))}
    </div>
  );
}
```

**src/index.css**

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f5f6f8; color: #1f2328; }
.app { max-width: 720px; margin: 40px auto; padding: 0 16px; }

.transfer { display: flex; gap: 16px; align-items: center; }

.panel {
  flex: 1; min-width: 0; background: #fff; border: 1px solid #d0d7de; border-radius: 8px;
  display: flex; flex-direction: column; height: 360px;
```

```
}
.panel__header {
  display: flex; justify-content: space-between; align-items: center;
  padding: 10px 12px; border-bottom: 1px solid #d0d7de;
}
.panel__select-all { display: flex; gap: 8px; align-items: center; cursor: pointer; }
.panel__title { font-weight: 600; }
.panel__count { font-size: 12px; color: #57606a; }
.panel__search { margin: 8px 12px; padding: 6px 8px; border: 1px solid #d0d7de; border-radius: 6px; }
.panel__list { list-style: none; margin: 0; padding: 0 4px 8px; overflow-y: auto; flex: 1; }
.panel__item { display: flex; gap: 8px; align-items: center; padding: 6px 8px; border-radius: 6px;
cursor: pointer; }
.panel__item:hover { background: #f3f4f6; }
.panel__item.is-checked { background: #ddf4ff; }
.panel__empty { margin: auto; color: #57606a; }

.controls { display: flex; flex-direction: column; gap: 8px; }
.controls__btn {
  width: 40px; height: 32px; border: 1px solid #d0d7de; border-radius: 6px;
  background: #fff; cursor: pointer; font-size: 16px;
}
.controls__btn:hover:not(:disabled) { background: #f3f4f6; }
.controls__btn:disabled { opacity: 0.4; cursor: not-allowed; }
.controls__btn:focus-visible, .panel__search:focus-visible { outline: 2px solid #0969da; outline-
offset: 2px; }

@media (max-width: 560px) {
  .transfer { flex-direction: column; align-items: stretch; }
  .controls { flex-direction: row; justify-content: center; }
}
```

## Reducer self-check ( `node selfcheck.mjs` )

```
import assert from 'node:assert/strict';
import { createInitialState, transferReducer as r, countChecked } from './transferReducer.js';

const A = { id: 'a', label: 'A' }, B = { id: 'b', label: 'B' }, C = { id: 'c', label: 'C' }, D = { id:
'd', label: 'D' };
let s = createInitialState({ left: [A, B, C], right: [D] });

s = r(s, { type: 'TOGGLE', id: 'a' });
s = r(s, { type: 'TOGGLE', id: 'c' });
s = r(s, { type: 'TOGGLE', id: 'd' });
assert.equal(countChecked(s.left, s.checked), 2);
assert.equal(countChecked(s.right, s.checked), 1);

// move selected right: only a, c move; d stays checked on the right
```

```
s = r(s, { type: 'MOVE', from: 'left', to: 'right', all: false });
assert.deepEqual(s.left.map((i) => i.id), ['b']);
assert.deepEqual(s.right.map((i) => i.id), ['d', 'a', 'c']);
assert.deepEqual([...s.checked], ['d']);

// no-op move returns same reference (no re-render)
const before = s;
assert.equal(r(s, { type: 'MOVE', from: 'left', to: 'right', all: false }), before);

// toggle twice = unchecked
s = r(s, { type: 'TOGGLE', id: 'b' });
s = r(s, { type: 'TOGGLE', id: 'b' });
assert.ok(!s.checked.has('b'));

// move all left: everything moves, checked cleared for moved items
s = r(s, { type: 'MOVE', from: 'right', to: 'left', all: true });
assert.deepEqual(s.left.map((i) => i.id), ['b', 'd', 'a', 'c']);
assert.equal(s.right.length, 0);
assert.equal(s.checked.size, 0);

// SET_MANY select / clear
s = r(s, { type: 'SET_MANY', ids: ['a', 'b'], value: true });
assert.equal(s.checked.size, 2);
s = r(s, { type: 'SET_MANY', ids: ['a'], value: false });
assert.deepEqual([...s.checked], ['b']);

// immutability: earlier state untouched
assert.deepEqual(before.right.map((i) => i.id), ['d', 'a', 'c']);
console.log('reducer self-check passed');
```

---

## 12. Wrap-up

### Edge cases handled

- Moving with nothing checked or from an empty list: buttons disabled **and** reducer no-op.
- Double clicks: no duplicates, because the reducer always reads the latest state.
- Moved items arrive unchecked, so the opposite button never lights up by surprise.
- Select-all respects the search filter and has a correct indeterminate state; an empty list is never "all checked".
- Hidden checked items survive filtering and are visible in the count.
- Duplicate labels, long labels, narrow screens, keyboard-only use.

### Trade-offs to mention

| Decision | Why | Cost |
|---|---|---|
| `useReducer` with one state object | Atomic moves, pure and testable | Slightly more code than `useState` for a tiny demo |
| One shared `checked` Set | Moves only delete ids; O(1) lookups | Every toggle re-renders both panels (fine at this size; `memo` per panel if needed) |
| Append moved items to the end | Simple, predictable, user sees "what I just added" at the bottom | Original order is lost; fix with an `order` index and a sort |
| Move includes hidden checked items | Selection is never silently dropped | Can surprise users; alternative is to move only visible checked items |
| Uncontrolled component ( `initial*` props) | Simple API for the interview | Parent cannot read or reset the lists; add `value` / `onChange` for a real form |
| Native checkboxes, not `role="listbox"` | Free keyboard + screen reader support | No arrow-key navigation or Shift-click range selection |

### What to add with more time

- **Tests:** React Testing Library for the table in section 2 (click item → click `>` → assert it is in the right list and unchecked).
- **Controlled mode:** `value={{ left, right }}` + `onChange` so it works inside forms.
- **Shift-click range selection** and arrow-key navigation ( `role="listbox"` + `aria-multiselectable` ).
- **Drag and drop** between lists, plus reordering within a list.
- **Keep original order** option (sort by original index on move).
- **Performance:** `React.memo` on `TransferPanel` , virtualization ( `react-window` ) for 10k+ items, debounce search.
- **Persistence / API:** load from server, save `rightIds` with optimistic update and rollback on failure.

---

## 13. Counter questions the interviewer may ask

### Requirements

#### Q1. Why did you ask whether moved items should stay checked?

Both are valid UX. Unchecking is the common choice (MUI, Ant Design) because if they stay checked, the opposite "move selected" button lights up and one wrong click undoes the move. It also changes the reducer: we must delete the moved ids from `checked` .

#### Q2. What if the interviewer says "keep the original order" when items move back?

Store an `order` (original index) on each item, or keep a master `allIds` array. On move, rebuild the target as `allIds.filter(id => targetIds.has(id))` . Moves stay O(n), and order is always the source order.

### Q3. What if they want only one-way transfer (pick from a pool, cannot move back)?

Hide the `<` and `≪` buttons via a prop like `oneWay`. The reducer does not change, which shows the value of one generic `MOVE` action.

## State design

### Q4. Why one `checked` Set instead of `leftChecked` and `rightChecked`?

Ids are unique across lists, so "checked in left" is `left ∩ checked`. With two sets, a move must remove ids from one set and decide about the other, which is two more places to get wrong. One set, derived intersections.

### Q5. Why not store `checked` as a boolean on each item (`{ id, label, checked }`)?

It mixes UI state into data. Every toggle would copy an item and its list, and you must strip the flag before sending data to an API. A Set of ids keeps data and selection separate and makes "clear selection" a one-liner.

### Q6. Why `useReducer` instead of three `useState`s?

A move is one event that changes three values. One reducer updates them together from the same snapshot, so they never drift, and the logic is a pure function I can test with `node:assert` without rendering anything.

### Q7. Why is the search `query` inside `TransferPanel` and not in `TransferList`?

Nothing outside the panel needs it. State lives in the lowest component that needs it: keeping it local means typing in one search does not re-render the other panel's subtree through new props.

## Challenge: atomic moves and set operations

### Q8. What happens if the user double-clicks `>` very fast?

Each click dispatches `MOVE`. React runs the reducer with the latest state, so the second call sees no checked items in `left`, hits `moving.length === 0` and returns the same state object. No duplicates, and React skips the re-render.

### Q9. What is the time complexity of a move?

O(n + k): one pass over the source with O(1) `Set.has`, plus copying k moved items. With `array.includes` instead of a Set it would be O(n·k), which is noticeable at thousands of items.

### Q10. Why return `state` instead of `{ ...state }` in the no-op case?

Returning the same reference tells React nothing changed, so it bails out of rendering. A new object, even with identical content, forces a re-render of the whole tree.

## Challenge: Set immutability

### Q11. You clicked a checkbox and nothing happened. What is the first thing you check?

Whether the Set (or array) was mutated in place. React compares references with `Object.is`; `checked.add(id)` then returning the same Set looks unchanged. Fix: `new Set(old)`, change the copy, return it.

## Challenge: select-all + indeterminate + search

### Q12. Why can't you write `<input indeterminate={true} />`?

`indeterminate` is a DOM property with no HTML attribute, and React does not map it. Set it imperatively: a ref plus `useEffect` that writes `ref.current.indeterminate`.

### Q13. With a search active, should select-all select hidden items?

No. The user can only judge what they see; selecting hidden items is a silent action. We pass only `visible` ids to `SET_MANY`. Items checked before the search stay checked, and the count shows them.

### Q14. Why `visible.length > 0 &&` in `allVisibleChecked`?

Without it, an empty list gives `0 === 0`, so select-all shows as checked on an empty list. The guard plus `disabled` keeps the empty state honest.

## Implementation steps

### Q15. Why extract `TransferPanel` only at step 7 and not at the start?

At the start I don't know the right props. After writing the markup twice, the shared shape is obvious (`title`, `items`, `checked`, `dispatch`). Extracting from real duplication avoids guessing an abstraction that turns out wrong.

### Q16. Why pass `dispatch` to the panel instead of `onToggle` / `onToggleAll` callbacks?

`dispatch` is stable (same reference every render) and avoids two wrapper callbacks. The trade-off is that the panel knows action names. For a public reusable component I would pass `onToggle(id)` and `onToggleMany(ids, value)` instead, so the panel works with any state shape.

### Q17. Why use `<label>` around each checkbox?

Clicking the text toggles the checkbox (bigger hit area), the screen reader reads the label with the checkbox, and Space works out of the box. No custom `onClick` or `onKeyDown` needed.

## Performance and scale

### Q18. What if each list has 1 million items?

Rendering is the bottleneck, not the reducer. Virtualize each list (`react-window`) so only ~20 rows are in the DOM. Debounce search and maybe run the filter in a Web Worker. Keep `checked` as a Set. For "move all", consider moving whole arrays by reference instead of copying item by item.

### Q19. Toggling one item re-renders both panels. Is that a problem?

At hundreds of items, no. If profiling shows it is, wrap `TransferPanel` in `React.memo`; it still re-renders because `checked` is a new Set. The fix is to pass each panel only its own checked ids, or to memo each row with `isChecked` as a boolean prop.

### Q20. What if the list data comes from a slow API?

Show skeleton rows and disable all buttons until loaded. Add an error state with retry. For saving moves, update the UI optimistically, send `rightIds`, and on failure roll back to the previous state (the reducer makes keeping the previous state trivial) and show a toast.

## Trade-offs and "why not X?"

### Q21. Why not use `role="listbox"` with `aria-multiselectable`?

It needs a full keyboard model (arrow keys, Space, Shift+arrows, `aria-activedescendant`) written by hand. Native checkboxes give correct keyboard and screen reader behavior for free in a 60-minute round. I would mention listbox as the upgrade if arrow-key navigation is required.

### Q22. How would you make it a controlled component?

Accept `value = { left, right }` and `onChange(next)`. Keep `checked` and `query` inside (they are UI state), and call `onChange` with the result of the same reducer logic instead of storing the lists locally.

### Q23. How would you test this?

Unit-test the reducer (done with node asserts). Then React Testing Library: render, `click` "Python", `click` the button by its `aria-label` "Move selected to right", assert Python is inside the "Selected" region and its checkbox is unchecked, and that the button is disabled again.